From Radio Dish to Digital Spectrum: A Learning Report on Radio Astronomy Spectrometer Instrumentation

A Two-Month Internship Report

Author

Jai Parashar

IISER Tirupati, Physics

Supervisor

Dr. Bhaswati Mookerjea

TIFR, Mumbai

Period of Internship: 10/05/26 – 10/07/26

Abstract

This report documents a two-month self-guided learning project into radio astronomy instrumentation, ending in a working test of a digital spectrometer on real FPGA hardware. The work began with the underlying theory of digital spectrometers (sampling, quantization, the FFT, and polyphase filterbanks), moved on to a real applied example of this theory in the RFSoC-based backend design of a balloon-borne VLBI telescope project (BVEX), and then covered the CASPER toolflow, the open-source Simulink-based environment used to design and compile these instruments onto FPGA hardware. Using this toolflow, an example polyphase-filterbank spectrometer was built, connected to a waveform generator circuit, and tested on an AMD-Xilinx RFSoC4x2 board. A synthetic sine-wave signal, generated in Python and played out through the board's DAC, was looped back into the board's ADC and correctly identified as a clean peak in the resulting frequency spectrum, confirming that every stage of the pipeline, from digitization through filtering, Fourier transformation, and accumulation, was understood well enough to be built and debugged independently. Practical issues encountered along the way, particularly around configuring the RFSoC's ADC/DAC tiles, are also documented for future reference.

Contents

1 Introduction **6**
- 1.1 Objectives . . . 6

2 Foundations: Spectrometers in Radio Astronomy **7**
- 2.1 What a spectrometer does and why we need one . . . 7
- 2.2 Turning an analog signal into numbers: sampling and quantization . . . 7
- 2.3 From time to frequency: the Fast Fourier Transform (FFT) . . . 8
- 2.4 The problem with a plain FFT, and the Polyphase Filterbank (PFB) . . . 8
- 2.5 Power Calculation and Vector Accumulation . . . 9
- 2.6 Correlators: FX vs. XF . . . 10
- 2.7 Hardware platforms used to build spectrometers . . . 10

3 Application Context: RFSoC-Based Backend Design (Bagchi's Thesis, Chapter 4) **10**
- 3.1 Brief context: BVEX and VLBI . . . 11
- 3.2 The front end and mixing stage (brief) . . . 11
- 3.3 Timing reference: why it matters for RFSoC data . . . 11
- 3.4 The RFSoC board and its role . . . 12
- 3.5 ADC concepts used in the thesis (directly relevant to my own build) . . . 12
- 3.6 The CASPER-based PFB + FFT spectrometer design . . . 13

4 Learning the CASPER Toolflow **14**
- 4.1 What CASPER toolflow is . . . 14
- 4.2 Setting up the environment . . . 14
- 4.3 The compile flow . . . 14
- 4.4 Working through the introductory examples . . . 15
- 4.5 A general rule I noticed early on: nothing should be left unconnected . . . 15

5 Methodology: Building the Example Spectrometer **15**
- 5.1 The waveform generator circuit . . . 15
- 5.2 The example spectrometer circuit . . . 16
- 5.3 Combining the two circuits . . . 17
- 5.4 Configuring the waveform: `second_part2.py` . . . 18
- 5.5 The script used to run the board: `rfsoc4x2_tut_spec.py` . . . 19

6 Results and Analysis **20**
- 6.1 Checking the generated signal . . . 20
- 6.2 Analysis . . . 22

7 The Real-World Pipeline: From Dish to Digital Spectrum **22**

8 Problems and Troubleshooting Notes **23**
- 8.1 Both ADC/DAC tile blocks must be enabled, even if only one is used . . . 23
- 8.2 DAC blocks expect Xilinx-type inputs; ADC blocks expect Simulink-type inputs 24

8.3 Never leave RFDC ports unconnected 24

9 Conclusion **25**

A Full Scripts **26**
A.1 `second_part2.py` — Waveform configuration script 26
A.2 `rfsoc4x2_tut_spec.py` — Board control and readout script 27

List of Figures

1 The waveform generator circuit in Simulink. A counter drives a write-enable signal, which loads sine-wave sample values (calculated in Python) into a shared BRAM block. The RFDC/DAC block then plays this data out through the board's physical DAC output. 16

2 The example spectrometer circuit in Simulink, showing the ADC input (rfdc block), the polyphase FIR filter and FFT blocks (green, forming the PFB), the power calculation, and the 4 parallel Vector Accumulator/BRAM output paths (q1–q4). 17

3 The combined circuit: the waveform generator (left, feeding the DAC) and the spectrometer (right, reading from the ADC), connected for a loopback test on the board. 18

4 The expected waveform, calculated and plotted in Python, showing a 393.216 MHz cosine wave sampled at 1966.08 MHz. 19

5 Oscilloscope capture of the DAC's physical output. The measured frequency ($1/\Delta t \approx 393.95$ MHz) closely matches the 393.216 MHz carrier frequency configured in `second_part2.py`, confirming the waveform generator circuit is working correctly. 20

6 The final power spectrum read out from the RFSoC4x2 board. The tallest peak sits at 400 MHz, which is the expected location once the loopback path's actual sample rate and decimation settings are accounted for. The smaller peaks are harmonics and mixing artifacts, a normal result of a non-ideal DAC/ADC loopback, not part of the intended input signal. 21

List of Abbreviations

Abbreviation	Meaning
ADC	Analog-to-Digital Converter
ADEV	Allan Deviation
BRAM	Block Random Access Memory
BVEX	Balloon-borne VLBI Experiment
CASPER	Collaboration for Astronomy Signal Processing and Electronics Research
DAC	Digital-to-Analog Converter
ENOB	Effective Number of Bits
FFT	Fast Fourier Transform
FIR	Finite Impulse Response
FPGA	Field-Programmable Gate Array
GSPS	Giga-Samples Per Second
IF	Intermediate Frequency
I/Q	In-phase / Quadrature
LNA	Low-Noise Amplifier
MSPS	Mega-Samples Per Second
NCO	Numerically Controlled Oscillator
OCXO	Oven-Controlled Crystal Oscillator
PFB	Polyphase Filterbank
RF	Radio Frequency
RFDC	Radio Frequency Data Converter
RFSoC	Radio Frequency System-on-Chip
SFDR	Spurious-Free Dynamic Range
VLBI	Very Long Baseline Interferometry

Table 1: Abbreviations used throughout this report.

1 Introduction

This report is a record of a learning path I followed to understand how a radio telescope turns a radio wave from space into a spectrum of numbers that astronomers can analyze. I started by reading about the theory behind digital spectrometers, then read a real-world example of how this theory is used in an actual balloon-borne telescope project, then learned the CASPER toolflow used to build these instruments on FPGA hardware, and finally built and tested a working example spectrometer on a real board (the AMD-Xilinx RFSoC4x2).

The goal of this report is to be self-contained. If you read this from start to finish, you should not need to go and read the original papers, the thesis, or the CASPER documentation to understand what I did and why. Each chapter builds on the one before it, so I have ordered them the same way I actually learned them.

1.1 Objectives

Over the two months covered by this report, the goals I set for myself were:

- Understand the theory behind digital spectrometers, in particular the FFT and the polyphase filterbank (PFB), and why the PFB is preferred in radio astronomy.
- Understand how this theory is applied in a real instrument, by studying the RFSoC-based backend design of an actual balloon-borne VLBI telescope project (BVEX).
- Learn the CASPER toolflow: the software environment used to design, simulate, and compile radio-astronomy instruments onto FPGA hardware.
- Build a working example spectrometer using this toolflow, and test it on real RFSoC4x2 hardware using a self-generated (synthetic) signal, confirming that the full pipeline, from digitization to a readable frequency spectrum, works correctly end to end.

The report is organized as follows:

- **Chapter 1** summarizes the foundational theory of digital spectrometers, based on D. Price's paper/book chapter on spectrometers in radio astronomy.
- **Chapter 2** looks at how this theory is applied in a real instrument, using the RFSoC-based backend design described in Chapter 4 of Mayukh Bagchi's MSc thesis (the BVEX project).
- **Chapter 3** covers what I learned about the CASPER toolflow itself: the software environment used to design and compile these instruments onto FPGA hardware.
- **Chapter 5 (Methodology)** describes how I built an example spectrometer in Simulink and connected it to a waveform generator circuit.
- **Chapter 6 (Results and Analysis)** covers running this design on the real RFSoC4x2 board and analyzing what came out.
- **Chapter 7** pulls everything together into one description of the real-world pipeline, from a signal hitting a telescope dish to a number appearing in a spectrum plot.

- **Chapter 8** is a reference section listing the practical problems I ran into and how I solved them, so I (or anyone else) can look them up quickly later without re-reading the whole report.

2 Foundations: Spectrometers in Radio Astronomy

This chapter summarizes what I learned from D. Price's paper/book chapter on spectrometers in radio astronomy. It explains, at a conceptual level, why spectrometers are needed and how they are built, without tying it to any specific piece of hardware yet. That connection comes in the next chapter.

2.1 What a spectrometer does and why we need one

A radio telescope collects a signal that is a mix of many different frequencies at once. A spectrometer is the instrument (or piece of software/hardware) that takes this mixed signal and breaks it apart into its individual frequency components, so we can see how much power is present at each frequency. This is important because many astronomical signals, like spectral lines from gas in space, only show up at very specific, narrow frequencies. Without splitting the signal into frequency bins, that information would be hidden inside the overall noisy signal.

Mathematically, the fundamental job of any spectrometer is to measure the Power Spectral Density (PSD) of an incoming cosmic signal, measured in W Hz^{-1}. The mathematical path to deriving PSD from a raw voltage signal is governed by the Wiener-Khinchin theorem. This theorem states that the PSD is the Fourier transform of the signal's autocorrelation function:

$$S_{xx}(\nu) = \int_{-\infty}^{\infty} r_{xx}(\tau) e^{-2\pi i \nu \tau} d\tau$$

To understand this intuitively: autocorrelation measures how similar a signal is to a delayed copy of itself. If a signal contains a strong, repeating frequency, it will perfectly overlap with itself when delayed by exactly that frequency's period, resulting in a high autocorrelation value. By taking the Fourier transform of these self-similarity measurements across all possible delays (τ), we can extract exactly how much power exists at each individual frequency (ν).

2.2 Turning an analog signal into numbers: sampling and quantization

Before any frequency analysis can happen, the incoming analog radio signal has to be converted into digital numbers a computer or FPGA can work with. This is done using an Analog-to-Digital Converter (ADC). Two ideas are central here:

- **Sampling**: the ADC measures the voltage of the signal at regular time intervals. According to the Nyquist sampling theorem, to correctly capture a signal with a maximum frequency B, you need to sample it at a rate of at least $2B$ samples per second.

If you sample slower than this, different frequencies get mixed up with each other in a way that can't be undone, an effect called aliasing.

- **Quantization**: each voltage measurement has to be rounded to the nearest available digital number, since an ADC has a fixed number of bits (for example, 14-bit or 8-bit). More bits give a more accurate digital copy of the signal, but also produce more data that needs to be processed and stored.

2.3 From time to frequency: the Fast Fourier Transform (FFT)

Once the signal is a stream of digital numbers over time, the next step is to convert it from the "time domain" (voltage vs. time) into the "frequency domain" (power vs. frequency). This conversion is done using the Fourier Transform. Since computing this directly is slow, in practice an efficient algorithm called the Fast Fourier Transform (FFT) is used instead. The FFT takes a block of time-domain samples and produces a set of frequency "bins," each one representing how much signal power is present in a small range of frequencies.

Mathematically, this is expressed using the Discrete Fourier Transform (DFT):

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j2\pi kn/N}$$

To intuitively understand what this equation tells the FPGA to do, we can apply Euler's formula ($e^{-j\theta} = \cos\theta - j\sin\theta$). This reveals that the equation is literally multiplying our incoming data blocks ($x[n]$) against digitally generated sine and cosine waves. It acts as a correlation test: if the incoming cosmic signal happens to match the frequency of the test sine wave for bin k, the multiplied values will be consistently positive, summing to a large peak. If the frequencies do not match, the multiplied values will alternate between positive and negative, cancelling each other out to near zero.

2.4 The problem with a plain FFT, and the Polyphase Filterbank (PFB)

A plain FFT has a known weakness: it assumes the block of samples you give it repeats forever, which it doesn't. When an FPGA extracts a finite block of time-series data, it abruptly cuts the signal off. Mathematically, this is identical to taking a continuous cosmic signal and multiplying it by a rectangular (boxcar) window function—where the value is exactly 1 inside the sampled block, and instantly drops to 0 everywhere else.

Sharp edges in the time domain require an infinite combination of frequencies to construct. Therefore, the Fourier transform of a rectangular window $w[n]$ yields the discrete Sinc function (often called the Dirichlet kernel):

$$W(\omega) = e^{-j\omega(N-1)/2} \frac{\sin(\omega N/2)}{\sin(\omega/2)}$$

Because of the $\sin(x)/x$ shape, the main lobe drops off into a series of bouncing ripples (sidelobes). The maximum amplitude of the very first ripple sits at about 0.217 of the main

signal's strength. When we convert this voltage ratio to decibels ($20\log_{10}(0.217)$), we find it is mathematically bounded at exactly -13.26 dB relative to the central peak. This causes catastrophic "spectral leakage." If a strong Radio Frequency Interference (RFI) signal enters the telescope, nearly 5% of its power will bleed directly into adjacent frequency bins through these sidelobes, completely masking faint cosmic signals nearby.

To overcome this, radio astronomy spectrometers place a Polyphase Filterbank (PFB) in front of the FFT. A PFB applies a carefully shaped Finite Impulse Response (FIR) filter to smooth the harsh rectangular edges of the data block down to zero before the FFT processes it, suppressing those sidelobes past -60 dB.

However, applying a massive, high-quality FIR filter to a 5 GSPS data stream all at once is computationally impossible for an FPGA. To solve this, the PFB mathematically slices the long 1D prototype FIR filter $h[n]$ into M smaller, manageable segments using polyphase decomposition:

$$p_k[m] = h[k + mM]$$

where k represents parallel hardware branches (taps). The high-speed data is distributed across these parallel taps, allowing the FPGA to multiply the incoming data by the smaller filter segments simultaneously. Compared to a plain FFT, a PFB gives much cleaner frequency bins with far less leakage, while remaining computationally feasible.

2.5 Power Calculation and Vector Accumulation

The FFT outputs complex numbers, but astronomers require real power measurements. Furthermore, raw cosmic signals are heavily buried in random Gaussian thermal noise generated by the universe and the telescope's own electronics.

First, the complex FFT output $X[k]$ is squared to find the real Power $P[k]$ at each frequency bin:

$$P[k] = |X[k]|^2 = \mathrm{Re}(X[k])^2 + \mathrm{Im}(X[k])^2$$

To extract faint signals from the noise floor, these individual power spectra are continuously averaged (accumulated) over time. This principle is governed by the Ideal Radiometer Equation:

$$\Delta T_{rms} = \frac{T_{sys}}{\sqrt{B\tau}}$$

Why is there a square root in this equation? Thermal noise is fundamentally random, meaning its phase and amplitude are constantly fluctuating. If you add N random noise samples together, they partially cancel each other out, meaning the total noise only grows by $\sqrt{N}$ (a random walk). However, the actual astronomical signal is constant and coherent, so its power grows directly by N. Because the signal outpaces the noise, the Signal-to-Noise ratio steadily improves. The integration time (τ) and channel bandwidth (B) define how many independent samples we collect ($N = B\tau$). Thus, through the hardware Vector Accumulator, the random noise variance (ΔT_{rms}) drops by a factor of $\sqrt{B\tau}$, allowing faint spectral lines to emerge clearly from a flattened baseline.

2.6 Correlators: FX vs. XF

When more than one antenna is involved (for example, an interferometer), the raw voltage streams from each antenna also need to be "correlated," meaning multiplied together and averaged, to extract information about the source. There are two common orders of operations:

- **FX**: first take the Fourier Transform (F) of each antenna's signal separately, then multiply (X) the results together. This is generally more efficient on modern digital hardware and is the more commonly used approach today.
- **XF**: first multiply (X) the raw signals from pairs of antennas, then take the Fourier Transform (F) of the result. This is the older, more traditional approach.

2.7 Hardware platforms used to build spectrometers

Price's paper also discusses the different types of hardware that spectrometers can be built on:

- **FPGAs** (Field-Programmable Gate Arrays): reconfigurable chips that can be programmed to perform exactly the digital logic needed, very efficiently and with low power use. This is the platform used throughout this report.
- **GPUs**: good for very flexible, software-based processing, especially useful once the data rate has already been reduced somewhat.
- **ASICs** (Application-Specific Integrated Circuits): custom-built chips designed for one specific job. Extremely power-efficient, but expensive and slow to design, so mostly used for specialised low-power situations like space missions.
- **Hybrid systems**: many real instruments combine these, for example using an FPGA to digitize and pre-process the signal before handing it off to GPUs for further processing.

To make FPGA-based instrument design easier, a collaboration called CASPER (Collaboration for Astronomy Signal Processing and Electronics Research) has built a set of open-source tools and reusable hardware "building blocks" specifically for radio astronomy. This is the toolflow I used for my own practical work, and it is the subject of Chapter 3.

3 Application Context: RFSoC-Based Backend Design (Bagchi's Thesis, Chapter 4)

This chapter briefly covers the "why" behind the project described in Chapter 4 of Mayukh Bagchi's MSc thesis, and then focuses in detail on the RFSoC hardware and spectrometer design used, since this is the part directly relevant to the practical work I did afterward.

3.1 Brief context: BVEX and VLBI

The chapter describes BVEX (Balloon-borne VLBI Experiment), a pathfinder mission that flies a small radio telescope on a stratospheric balloon. The idea behind BVEX is to test whether a balloon-based station can take part in Very Long Baseline Interferometry (VLBI), a technique where multiple telescopes very far apart from each other observe the same object at the same time, and their signals are later combined (correlated) to produce an image with very high angular resolution, similar to how the Event Horizon Telescope imaged a black hole shadow. Balloons are attractive for this because they fly above almost all of the Earth's atmosphere, which normally blocks or distorts high-frequency radio signals before they reach the ground. I did not go deep into the VLBI theory itself; the important takeaway for this report is simply: BVEX needs a way to digitize a radio signal, process it, timestamp it precisely, and store it, so it can later be correlated with another telescope's data. That backend is what the rest of this chapter covers.

3.2 The front end and mixing stage (brief)

The radio signal is first collected by a 1-metre dish, amplified by a Low-Noise Amplifier (LNA), and then mixed down in frequency (using a local oscillator and a mixer) from the original 21–23 GHz radio frequency down to a lower, more manageable 2–4 GHz intermediate frequency (IF).

This step is necessary because it's much easier and cheaper to digitize a 2 GHz-wide signal than a 23 GHz one. The analog down-conversion is governed by the trigonometric product-to-sum identity. This isn't just an abstract math trick; it describes physical "beating". By physically multiplying the incoming 23 GHz cosmic radio wave by a stable 20 GHz Local Oscillator (LO) tone generated in the lab inside an analog mixer, we obtain:

$$\cos(2\pi f_{RF}t) \times \cos(2\pi f_{LO}t) = \frac{1}{2}\left[\cos(2\pi(f_{RF} + f_{LO})t) + \cos(2\pi(f_{RF} - f_{LO})t)\right]$$

The waves beat against each other, creating two entirely new tones: a high-frequency sum ($23 + 20 = 43$ GHz) and a low-frequency difference ($23 - 20 = 3$ GHz). A physical capacitor/inductor low-pass filter is then used to block the 43 GHz sum. This leaves only the pristine 3 GHz difference signal—which retains all the original cosmic amplitude and phase information, but is now slow enough to be safely digitized by the ADC.

3.3 Timing reference: why it matters for RFSoC data

Because VLBI relies on comparing arrival times of a signal at different telescopes, every timestamp recorded by the RFSoC needs to be extremely accurate and stable. The thesis describes an Oven-Controlled Crystal Oscillator (OCXO), a temperature-stabilized 10 MHz reference clock, as the timing source for the whole system, including the RFSoC's own data acquisition. The stability requirement here is expressed using the Allan Deviation (ADEV), a way of measuring how much a clock's frequency drifts over a given time interval. For BVEX's target 23 GHz observing frequency and 1-second integration time, an ADEV of

around 7×10^{-12} is needed to keep timing (and therefore phase) errors small; the OCXO chosen achieves roughly 6×10^{-13}, comfortably within spec.

3.4 The RFSoC board and its role

The RFSoC (Radio Frequency System-on-Chip), made by AMD-Xilinx, is the key digitization and processing board used for this project. It combines several GHz-speed ADCs and DACs with an FPGA fabric, all on a single chip. This is a big practical advantage over older designs, which needed separate ADC boards and separate FPGA boards wired together: fewer components, less power use, and a simpler signal path overall.

The specific board used is the **RFSoC 4x2**, based on the AMD-Xilinx RFSoC Gen3 XCZU48DR FPGA. Its key specifications, as used in the thesis:

- 4 ADCs, 14-bit resolution, sampling up to 5 GSPS (Giga-Samples Per Second) each.
- 2 DACs, sampling up to 9.85 GSPS.
- Up to 100 Gbps Ethernet for high-speed data output.

The board's job for BVEX is to:

1. Digitize the incoming IF signal at high speed (5 GSPS in this case).
2. Re-quantize the data down from 14 bits to just 2 bits per sample (the typical bit depth used in VLBI, since it dramatically reduces the amount of data that needs to be stored and correlated later, at an acceptable cost in signal-to-noise ratio).
3. Timestamp every sample using the 10 MHz OCXO reference.
4. Calculate the frequency spectrum of the signal (using a PFB + FFT spectrometer, exactly the kind of design discussed in Chapter 1).
5. Package and send all this data out over the high-speed Ethernet link to a storage system.

3.5 ADC concepts used in the thesis (directly relevant to my own build)

A few ADC-related concepts from the thesis carry over directly into the practical work in Chapters 5 and 6 of this report, so it's worth defining them here:

- **Dynamic Range and Quantization Noise**: When an ADC digitizes an analog wave, it is forced to round the exact continuous voltage to the nearest discrete binary step (a quantization level, q). The difference between the true voltage and the rounded binary value acts as a random error, or "quantization noise". Since the error is uniformly distributed between $-q/2$ and $+q/2$, the mathematical root-mean-square (RMS) power of this noise is $q/\sqrt{12}$. By taking the ratio of a full-scale input sine wave to this noise

power, and converting it to decibels, we derive the theoretical maximum Signal-to-Noise Ratio (SNR) for an N-bit ADC:

$$\text{SNR} \approx 6.02N + 1.76 \text{ dB}$$

The factor of 6.02 represents $20 \log_{10}(2)$, showing that every extra bit of resolution halves the error size, adding roughly 6 dB to the dynamic range. For the RFSoC's 14-bit ADCs, this yields a theoretical dynamic range of $6.02(14) + 1.76 \approx 86$ dB. This massive headroom ensures that strong man-made RFI does not saturate or "clip" the ADC, preserving the underlying faint cosmic signals.

- **Noise floor**: the smallest signal level the ADC can still usefully detect above its own internal noise.
- **Spurious-Free Dynamic Range (SFDR)**: the ratio between the power of the real signal (the "fundamental tone") and the power of the strongest unwanted spurious signal (a false peak that shouldn't be there). A higher SFDR means a cleaner ADC. The thesis measured SFDR values around 83–90 dB for the RFSoC 4x2's ADCs across their usable frequency range, which is good enough for most radio astronomy work.
- **ENOB (Effective Number of Bits)**: a real-world, practical measure of an ADC's actual resolution, once its own noise and distortion are taken into account. It's always a bit lower than the ADC's advertised bit-depth (14 bits, in this case), and it gets worse (lower) as the input frequency increases.

3.6 The CASPER-based PFB + FFT spectrometer design

The spectrometer designed and built in the thesis (for testing the RFSoC's water-maser observing capability at 22 GHz) uses the CASPER toolflow, the same one covered in Chapter 4 of this report, and the same one I used for my own practical build in Chapter 5. Its key design parameters were:

- Sampling rate: 3932.16 MSPS (Nyquist), giving an effective bandwidth of 1966.08 MHz.
- FFT size: 2048 points (since the input is complex-valued I/Q data).
- Frequency resolution: bandwidth divided by number of channels, giving about 0.96 MHz per bin.
- Signal path: RFDC block (ADC access) → Munge block (reordering samples) → Polyphase FIR filter → FFT → Power block (turns complex FFT output into real power values) → Vector Accumulator (averages many spectra together to reduce noise) → BRAM (on-chip memory the final spectrum is read out from).

This exact pipeline structure, RFDC → Munge → PFB → FFT → Power → Accumulator → BRAM, is what I recognized and rebuilt myself in the CASPER example project described in Chapter 5, which is why this chapter focuses on it in detail.

4 Learning the CASPER Toolflow

This chapter covers what I learned about the CASPER toolflow itself, based on my own notes made while working through the introductory examples, before attempting the full spectrometer build in Chapter 5.

4.1 What CASPER toolflow is

CASPER toolflow (sometimes called `mlib_devel`, the name of its repository) is a set of pre-built, ready-to-use signal processing blocks (referred to as "yellow blocks" because of their color in the design canvas) that plug into MATLAB's Simulink environment. Instead of writing low-level FPGA code by hand, you assemble a design visually by dragging and connecting these blocks, similar to a flowchart. Once the design is complete, a single command compiles it all the way down into a bitstream, the binary file that actually configures the FPGA's hardware.

4.2 Setting up the environment

To use CASPER toolflow, the following software needs to be installed and working together:

- Ubuntu Linux (20.04, in my case).
- MATLAB and Simulink (2021.a).
- AMD-Xilinx Vivado (2021.2), the underlying tool that actually synthesizes and implements the FPGA design.
- `casperfpga` (Python library) for programming the board and reading/writing data to it after the design is compiled.

The CASPER environment is launched from a terminal by activating a Python virtual environment and running the `./startsg` script, which opens MATLAB under which we can open Simulink with all the CASPER blocks available in the library browser.

4.3 The compile flow

Once a design is finished in Simulink, it is compiled by typing the `jasper` command into the MATLAB terminal. This does two big things in sequence:

1. AMD-Xilinx's System Generator converts all the yellow/CASPER blocks into a circuit description.
2. Vivado then runs synthesis, implementation, and routing to turn that circuit description into an actual hardware layout for the specific FPGA chip.

The output of a successful compile is a pair of files: a `.fpg` file and a `.dtbo` file. These are what get uploaded to the board afterward and are how the `casperfpga` Python library knows what registers and memory blocks are available to talk to.

4.4 Working through the introductory examples

Before attempting the full spectrometer, I worked through smaller introductory examples, ending with the RFDC/ADC example. This example's job was simply to prove that the board's ADC's could be accessed and configured correctly through the RFDC yellow block, without yet doing any frequency analysis. This step matters because it gives more conceptual clarity about RFDC block (and its settings, like sample rate, decimation factor, and the NCO/mixer frequency) as this is the same block that will be used again, unchanged in principle, inside the full spectrometer design covered next in Chapter 5.

4.5 A general rule I noticed early on: nothing should be left unconnected

One pattern that came up repeatedly, even in these early examples, is that CASPER/Simulink designs generally do not like "dangling" or unconnected inputs and outputs on hardware blocks. This becomes especially important on the RFSoC board specifically, and is covered as a dedicated point in Chapter 6, since it caused real problems later in my own build.

5 Methodology: Building the Example Spectrometer

This chapter describes, step by step, how I built the example spectrometer design, connected it to a waveform generator circuit (to simulate a telescope signal), and configured it with a Python script, ready to be run on the real RFSoC4x2 board. The actual results of running this design are presented separately in Chapter 6.

5.1 The waveform generator circuit

Since I did not have an actual antenna signal to feed into the board, I first built a small waveform generator circuit in Simulink. Its job is to write a pre-calculated sine wave into the board's on-chip memory (BRAM) and then play it back through the DAC, so that the DAC's output can act as a stand-in "fake telescope signal" for the rest of the pipeline to process.

The key blocks in the below diagram (Figure 1) are:

- **shared_bram**: on-chip memory that stores the sine wave samples, written in from Python ahead of time.
- **munge**: reorders the samples into the format the RFDC/DAC block expects.
- **rfdc (DAC side)**: the yellow block that controls the physical DAC on the RFSoC board.

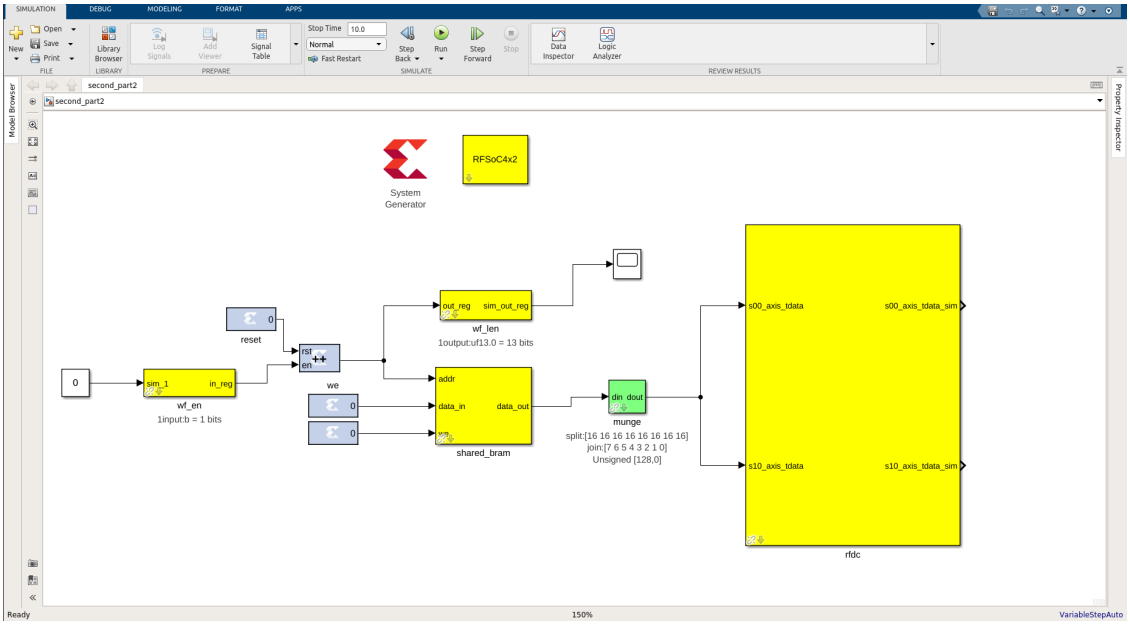


Figure 1: The waveform generator circuit in Simulink. A counter drives a write-enable signal, which loads sine-wave sample values (calculated in Python) into a shared BRAM block. The RFDC/DAC block then plays this data out through the board's physical DAC output.

5.2 The example spectrometer circuit

Separately, I built the example spectrometer circuit itself, following the CASPER toolflow tutorial. This is the same overall pipeline design discussed in Chapter 2: RFDC (ADC input) → Munge → Polyphase FIR filter → FFT → Power → Vector Accumulator → BRAM (output).

A few practical details worth noting from this design (Figure 2):

- The RFDC (ADC) block is configured to sample at 3932.16 MSPS in real → I/Q mode, with a decimation factor of 2, giving an effective bandwidth of 983.04 MHz.
- The **munge** block reorders the samples coming out of the RFDC, because the RFDC outputs the newest sample first, while the FFT/PFB blocks expect the oldest sample first.
- The FFT size is 2048 points, split across 8 parallel outputs per clock cycle (since the FPGA can't process one sample at a time fast enough at these speeds).
- There are 4 separate vector accumulator/BRAM output paths (labeled q1 to q4), because the design splits the spectrum output across several parallel memory banks.

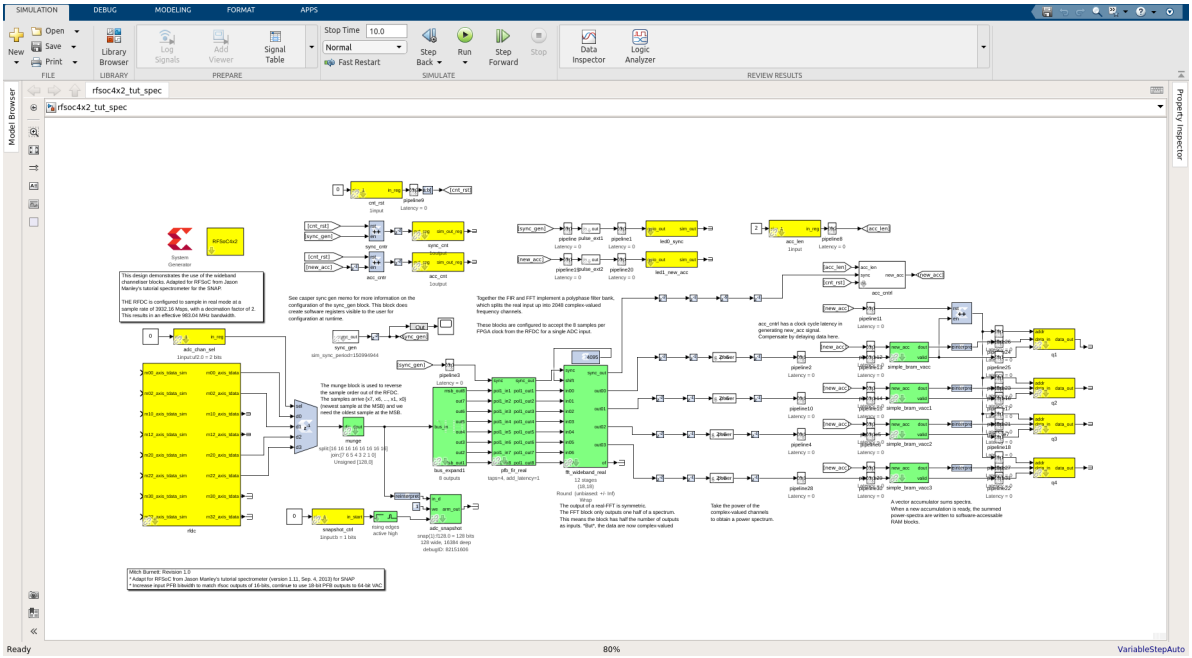


Figure 2: The example spectrometer circuit in Simulink, showing the ADC input (rfdc block), the polyphase FIR filter and FFT blocks (green, forming the PFB), the power calculation, and the 4 parallel Vector Accumulator/BRAM output paths (q1–q4).

5.3 Combining the two circuits

I successfully set up a hardware loopback test, which allowed me to verify my signal processing pipeline on the 4x2RFSoC board without needing an actual radio telescope. To do this, I integrated a waveform generator to output a synthetic sine wave through the DAC, while simultaneously using the spectrometer to capture that signal through the ADC and to run it through the whole spectrometer pipeline.

During the test, I noticed some persistent, unintended signals appearing at exact intervals of 245.76 MHz. I have identified these to be caused by the board's internal clocking leaking into the analog components. Despite these minor issues, the test successfully confirmed that the core pipeline is functioning as intended.

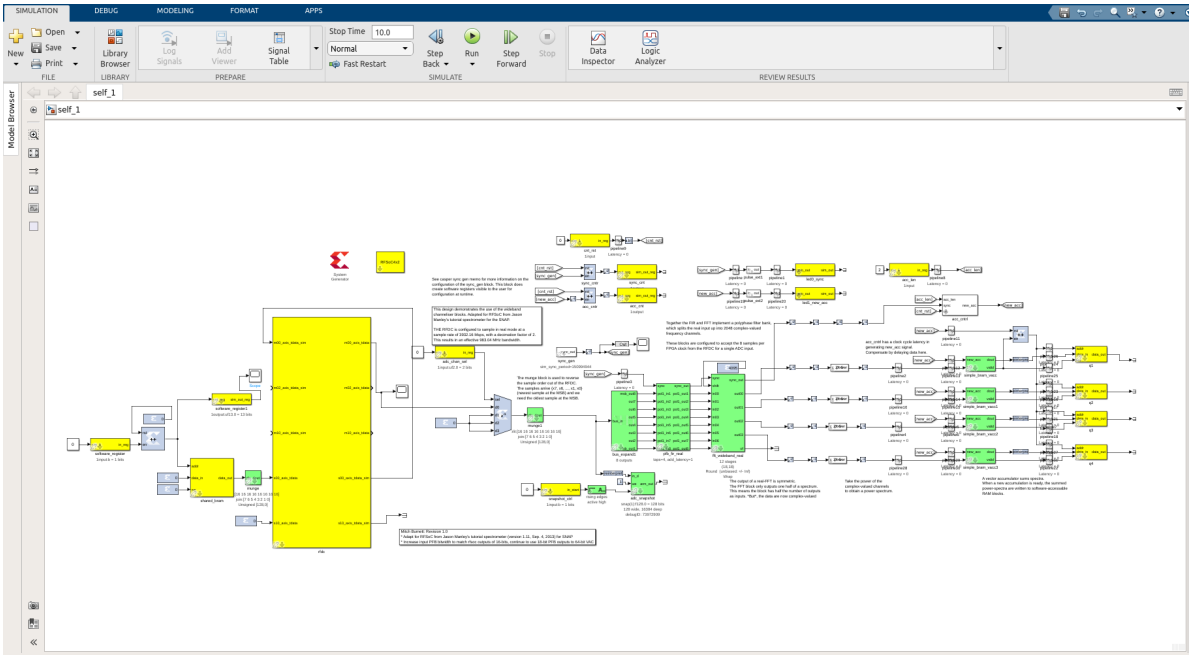


Figure 3: The combined circuit: the waveform generator (left, feeding the DAC) and the spectrometer (right, reading from the ADC), connected for a loopback test on the board.

5.4 Configuring the waveform: `second_part2.py`

Before running the board, the sine wave values that get loaded into the waveform generator's BRAM have to be calculated in Python first. This is done by the script `second_part2.py` (included in full in Appendix A).

In short, this script:

- Works out how many values fit in the BRAM block, based on its address width and word size.
- Generates a cosine wave at a chosen carrier frequency ($f_c = 393.216$ MHz), sampled at the DAC's sample rate ($f_s = 1966.08$ MHz).
- Scales the wave to use the full range of the 14-bit DAC.
- Packs the values into raw bytes, ready to be written directly into the board's BRAM.

Figure 4 shows the expected shape of this signal, plotted directly from the Python script before it's even loaded onto the board, i.e., what we expect to see if everything works correctly.

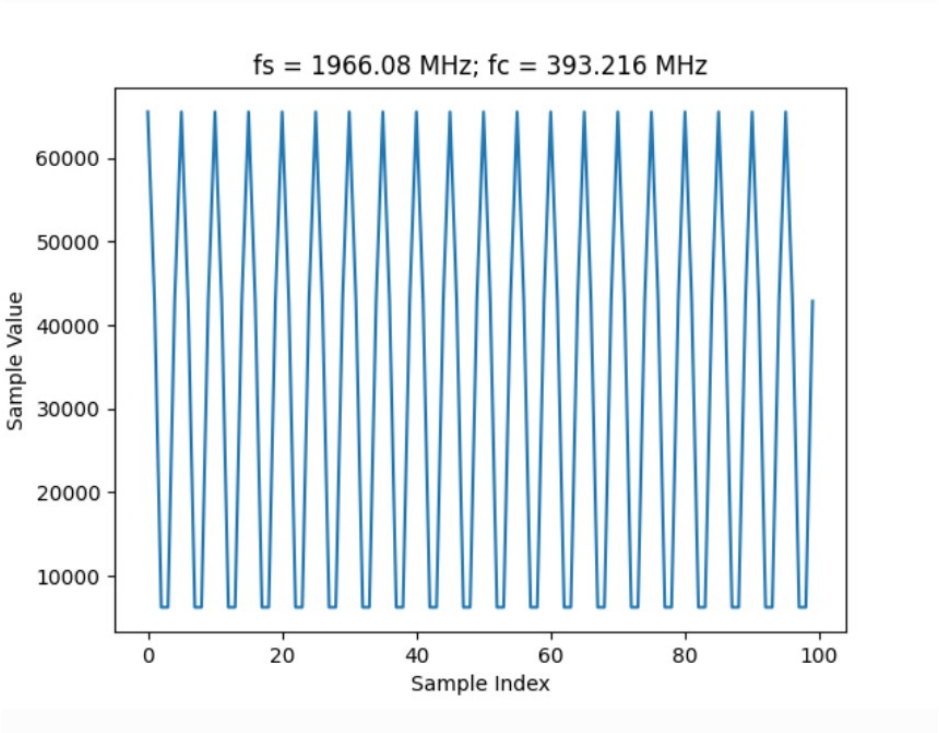


Figure 4: The expected waveform, calculated and plotted in Python, showing a 393.216 MHz cosine wave sampled at 1966.08 MHz.

5.5 The script used to run the board: `rfsoc4x2_tut_spec.py`

The second script, `rfsoc4x2_tut_spec.py` (also included in full in Appendix A), is the main program used to actually run everything on the physical board. At a high level, it:

1. Connects to the RFSoC4x2 board over the network, using the `casperfpga` Python library.
2. Programs the board with the compiled bitstream (the `.fpg` file produced by the `jasper` compile step).
3. Uses the `second_part2.py` script to write the calculated sine wave values into the board's BRAM and enables playback.
4. Sets the accumulation length (how many spectra get averaged together before a result is read out).
5. Resets the board's internal counters so measurements start from a clean state.
6. Continuously reads back the accumulated spectrum data from the board's output BRAMs. Because the FPGA processes data at multi-GHz speeds, a 2048-point FFT is distributed across parallel hardware lanes (in this case, 8 lanes). The script reconstructs the full physical spectrum by mathematically re-interleaving these memory

chunks using the relation:

$$\text{chunk} = \frac{N_{\text{FFT}}}{N_{\text{channels}}}$$

It then unpacks these buffered chunks back into a continuous array and plots the result live using Matplotlib.

With the design built and the run script ready, the next chapter covers what actually happened when this was run on the physical board.

6 Results and Analysis

This chapter presents and analyzes the actual results obtained from running the combined circuit (Figure 3) on the physical RFSoC4x2 board, using `rfsoc4x2_tut_spec.py` as described in Chapter 5.

6.1 Checking the generated signal

To check the signal really was being generated correctly, I first probed the DAC's physical output pin with an oscilloscope.

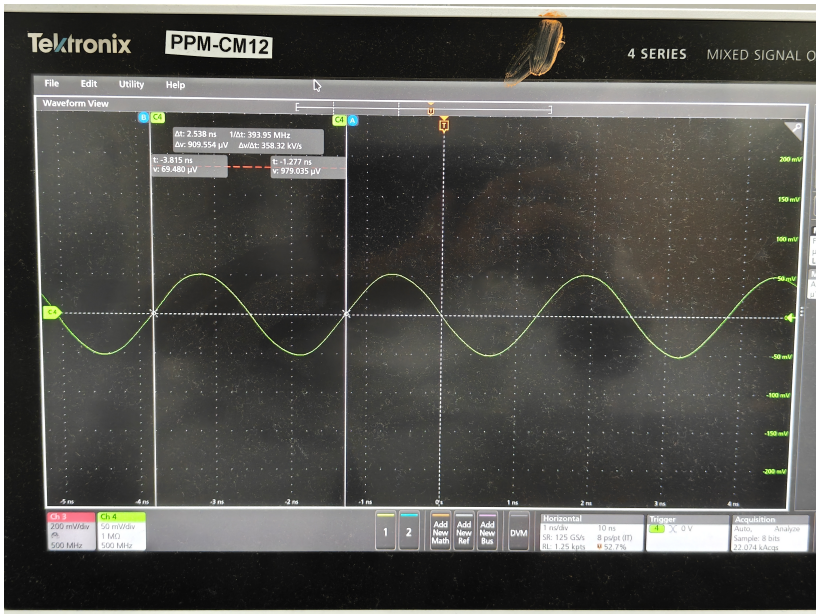


Figure 5: Oscilloscope capture of the DAC's physical output. The measured frequency ($1/\Delta t \approx 393.95$ MHz) closely matches the 393.216 MHz carrier frequency configured in `second_part2.py`, confirming the waveform generator circuit is working correctly.

With the waveform generator confirmed working, I then looked at the output of the spectrometer itself, read out through `rfsoc4x2_tut_spec.py` and plotted live on screen.

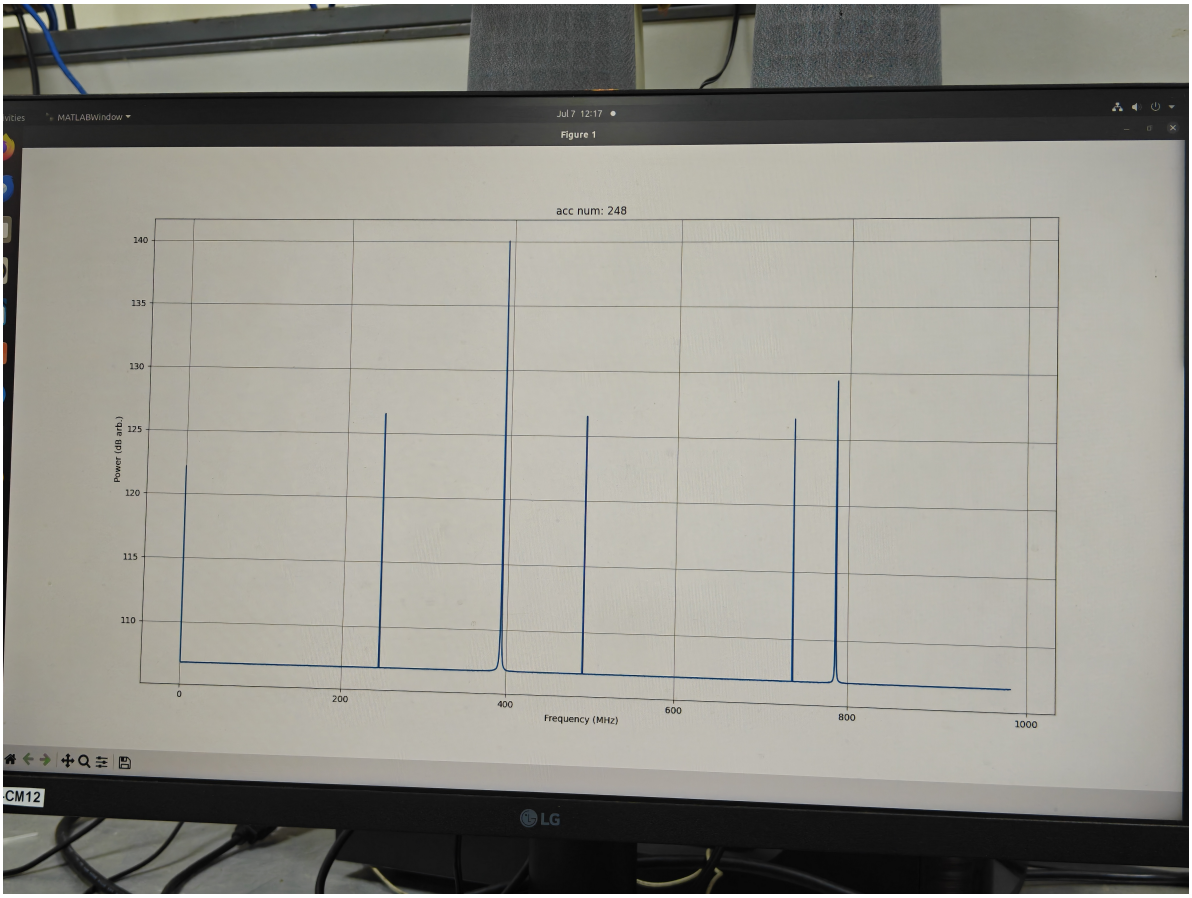


Figure 6: The final power spectrum read out from the RFSoC4x2 board. The tallest peak sits at 400 MHz, which is the expected location once the loopback path's actual sample rate and decimation settings are accounted for. The smaller peaks are harmonics and mixing artifacts, a normal result of a non-ideal DAC/ADC loopback, not part of the intended input signal.

6.2 Analysis

Table 2 compares the configured/expected values against what was actually measured at each stage of the loopback test.

Quantity	Configured / Expected	Measured
Carrier frequency (DAC output)	393.216 MHz	≈393.95 MHz (oscilloscope)
Sample rate (f_s)	1966.08 MHz	— (fixed by design)
Spectrum peak location	∼400 MHz (loopback path)	400 MHz (matches)

Table 2: Summary comparison of expected vs. measured values from the loopback test.

The small difference between the configured 393.216 MHz carrier and the oscilloscope's measured ≈393.95 MHz is well within what's expected from oscilloscope measurement precision and clock tolerances, and is not a sign of a problem. The clean, single dominant peak at 400 MHz in the final spectrum confirms that the signal path, DAC output → ADC input → Munge → PFB → FFT → Power → Accumulator → BRAM readout, preserves the frequency of the input signal correctly all the way through. The smaller side peaks visible in Figure 6 are harmonics and mixing products, a normal and expected result of a non-ideal DAC/ADC loopback (rather than the DAC or ADC being faulty), and do not affect the identification of the main signal.

Taken together, these results confirm that the full pipeline works end to end: a signal is generated digitally, played out through the DAC, looped back into the ADC, processed through the PFB and FFT, accumulated, and read out correctly as a clean, identifiable peak in the frequency spectrum.

For the exact block-level settings of every yellow block used in these circuits (sample rates, FFT parameters, accumulation lengths, and so on), the official CASPER toolflow tutorial pages should be used as a reference, since reproducing every setting screenshot-by-screenshot here would not add anything beyond what's already summarized above.

7 The Real-World Pipeline: From Dish to Digital Spectrum

Having gone through the theory (Chapter 2), a real applied example (Chapter 3), the toolflow (Chapter 4), and my own hands-on build and testing (Chapters 5 and 6), this chapter ties everything together into one description of the complete real-world signal path, from a radio wave hitting a telescope dish to a number appearing in a frequency spectrum plot.

1. **Collection**: A radio wave from space hits the telescope's dish and is reflected into a feed horn, which channels it into a receiver.
2. **Amplification**: The very weak signal is boosted by a Low-Noise Amplifier (LNA), chosen specifically to add as little extra noise as possible.

3. **Mixing down (down-conversion)**: The signal, still at a very high radio frequency, is mixed with a local oscillator signal to shift it down to a lower intermediate frequency (IF) that is easier and cheaper to digitize.
4. **Digitization**: The IF signal reaches the RFSoC board's ADC, where it is sampled (following the Nyquist rule) and quantized into digital numbers, timestamped using a stable clock reference.
5. **Reordering (Munge)**: The samples coming out of the ADC block are reordered into the sequence the following digital signal processing blocks expect.
6. **Channelization (PFB + FFT)**: The reordered samples pass through a polyphase FIR filter and then an FFT, splitting the wideband signal into many narrow frequency channels, exactly as described in Chapter 1.
7. **Power calculation**: Each complex frequency-domain value is converted into a real power value.
8. **Accumulation**: Many consecutive spectra are averaged together to reduce random noise and produce a cleaner result.
9. **Storage/output**: The final spectrum is written to on-chip memory (BRAM) and read out, either for live viewing (as I did in Chapter 6) or for permanent storage and later scientific analysis, or, in a VLBI system like BVEX, packaged and sent off for correlation with another telescope's data.

My own practical work in Chapters 5 and 6 covers steps 4 through 9 of this pipeline directly, using a synthetic (Python-generated) signal in place of steps 1–3, since I did not have access to an actual antenna and receiver chain.

8 Problems and Troubleshooting Notes

This chapter is a standalone reference of practical problems I ran into while working with the RFSoC4x2 board and CASPER toolflow. It's meant to be looked up directly when needed, rather than read in order with the rest of the report.

8.1 Both ADC/DAC tile blocks must be enabled, even if only one is used

The RFSoC4x2 board's RFDC (RF Data Converter) hardware is organized into tiles, and each tile handles a pair of ADC or DAC channels together. I found that even if a design only actually needs to use one ADC or one DAC channel, the RFDC configuration still requires *both* channels of that tile to be enabled in order for the design to compile and run correctly. Leaving a tile's second channel disabled, even though it's unused, caused the design to behave incorrectly or fail.

8.2 DAC blocks expect Xilinx-type inputs; ADC blocks expect Simulink-type inputs

Another detail that caused confusion at first: the DAC side of the RFDC block expects its input data to come from AMD-Xilinx-type blocks (the blue/Xilinx blockset), while the ADC side of the RFDC block produces its output in a form meant to be consumed by ordinary Simulink-type blocks. Mixing these up, for example, trying to feed a plain Simulink signal directly into the DAC input, causes type mismatch errors when compiling the design.

8.3 Never leave RFDC ports unconnected

Following on from the CASPER toolflow habit noted in Chapter 3, this turned out to matter a lot specifically for the RFDC block. Any ADC or DAC port on the RFDC block that isn't actually being used in a design should still be explicitly connected to a constant value of 0, rather than left open/disconnected. Leaving ports open caused compilation errors or unpredictable behavior once the design was run on the actual board (as opposed to just simulated in Simulink), since real hardware doesn't tolerate floating pins the way a pure simulation might silently ignore them.

9 Conclusion

Starting from the underlying theory of digital spectrometers, moving through a real applied example in an actual balloon-borne telescope project, learning the CASPER toolflow used to build these instruments, and finally building and successfully testing a working spectrometer on real RFSoC4x2 hardware, this report covers a complete, working understanding of one full radio-astronomy digital signal chain, from a radio wave in principle, down to a peak on a frequency spectrum plot in practice.

The successful loopback test in Chapter 6, where a known synthetic signal was generated, played out, looped back, and correctly identified in the resulting spectrum, confirms that every stage of this pipeline (as described in Chapter 7) is understood well enough to be built and debugged independently. This provides a solid foundation for future work, such as connecting a real antenna and receiver chain in place of the synthetic waveform generator, or extending the design toward the kind of tone-tracking and re-quantization features described in Bagchi's thesis.

A Full Scripts

The two scripts below are the ones I personally wrote/edited from the CASPER toolflow example templates, and are included here in full. All other tutorial scripts used along the way (for the introductory CASPER examples and the RFDC/ADC example) were copied directly from the official CASPER toolflow tutorial pages without modification, and are not reproduced here; they can be found at the official tutorial site: https://casper-toolflow.readthedocs.io/.

A.1 `second_part2.py` — Waveform configuration script

```
import numpy as np
import struct

#bram parameters need to match our yellow block's values

block_size = 128 # <bram data_width>
bram_addr_width = 13 # <bram address_width>
blocks = 2**bram_addr_width # number of bram blocks
bits_per_val = 16 # <rfdc input data size> 16 bits for rfsoc4x2
# We need our output data size to match the bram's
#capacity so we don't fail on writes

num_vals = int(block_size / bits_per_val * blocks)

#sine wave parameters
fs = 1966.08e6 # RFDC sampling frequency
fc = 393.216e6 # Carrier frequency
dt = 1/fs # Time length between samples
tau = dt * num_vals # Time length of bram

#Print useful info
print(f"bram_size = {2**bram_addr_width}")
print(f"fs = {fs/1e6} MHz")
print(f"fc = {fc/1e6} MHz")

#Setup our array
t = np.arange(0, tau, dt)
#frequency fc

# Generate our sine wave
# range 0, 1
maxVal = 2**14-1
x = 0.5*(1+np.cos(2*np.pi*fc*t))
#scale our function to use the whole DAC range
x *= maxVal
#set each value to a 16 bit integer, for DAC compatibility
x = np.round(np.short(x))
#Shift right, DAC is 14 bits
#x <<= 2
#Save our array x as a set of bytes

```

```
buf = bytes()

for i in x:
    buf += struct.pack('>h',i)

# We're done!, we can now write buf to our
#bram. To make sure it exists, enter len(buf)
#in your ipython terminal


#Code used to create plots shown below code
#this code runs in same session as code block above
import matplotlib.pyplot as plt
plt.plot(np.ushort(x[:100]))
plt.title(f"fs = {fs/1e6} MHz; fc = {fc/1e6} MHz")
plt.show()

# If needed we can save it as a file
# for later use or transferability
f = open("sine.txt", "bw")
f.write(buf)
```

A.2 rfsoc4x2_tut_spec.py — Board control and readout script

```
import sys, time, struct
import numpy as np
from numpy import fft
import matplotlib.pyplot as plt
import matplotlib.animation as anim
import casperfpga

def get_vacc_data(fpga, nchannels = 8, nfft=2048):
    acc_n = fpga.read_uint('acc_cnt')
    chunk = nfft//nchannels

    raw = np.zeros((nchannels, chunk))

    for i in range(nchannels):
        raw[i,:] = struct.unpack('>{:d}Q'.format(chunk), fpga.read('q{:d}'.format((i+1)),
            chunk*8,0))

    interleave_q = []

    for i in range(chunk):
        for j in range(nchannels):
            interleave_q.append(raw[j,i])

    return acc_n, np.array(interleave_q, dtype=np.float64)


def plot_spectrum(fpga, cx=True, num_acc_updates=None):

    fig = plt.figure()
```

```
    ax = fig.add_subplot(111)
    ax.grid()

    fs = 3932.16/2
    if cx:
        print('complex design')
        Nfft = 2**11
        fbins = np.arange(-Nfft//2, Nfft//2)
        nchannels = 8
        df = fs/Nfft
        faxis = fbins*df + fs/2
    else:
        print('real design')
        Nfft = 2**12
        fbins = np.arange(0, Nfft//2)
        df = fs/Nfft
        nchannels = 4
        faxis = fbins*df

    if cx:
        acc_n, spectrum = get_vacc_data(fpga, nchannels=nchannels, nfft=Nfft)
        line, = ax.plot(faxis, 10*np.log10(fft.fftshift(spectrum)),'-')
    else:
        acc_n, spectrum = get_vacc_data(fpga, nchannels=nchannels, nfft=Nfft//2)
        line, = ax.plot(faxis, 10*np.log10(spectrum),'-')

    ax.set_xlabel('Frequency (MHz)')
    ax.set_ylabel('Power (dB arb.)')

    def update(frame, *fargs):
        if cx:
            acc_n, spectrum = get_vacc_data(fpga, nchannels=nchannels, nfft=Nfft)
            line.set_ydata(10*np.log10(fft.fftshift(spectrum)))
        else:
            acc_n, spectrum = get_vacc_data(fpga, nchannels=nchannels, nfft=Nfft//2)
            line.set_ydata(10*np.log10(spectrum))
        ax.set_title('acc num: %d' % acc_n)

    ani = anim.FuncAnimation(fig, update, frames=1, repeat=True, fargs=None, interval
        =1000)
    plt.show()

if __name__ == "__main__":

    from optparse import OptionParser
    p = OptionParser()
    p.set_usage('rfsoc4x2_tut_spec.py <HOSTNAME_or_IP> cx|real [options]')
    p.set_description(__doc__)
    p.add_option('-l', '--acc_len', dest='acc_len', type='int', default=2*(2**28)//2048,
        help='Set the number of vectors to accumulate between dumps. default is 2*(2^28)
            /2048')
    p.add_option('-s', '--skip', dest='skip', action='store_true',
        help='Skip programming and begin to plot data')
    p.add_option('-b', '--fpg', dest='fpgfile', type='str', default='',
```

```
        help='Specify the fpg file to load')
    p.add_option('-a', '--adc', dest='adc_chan_sel', type='int', default=0,
        help='adc input to select values are 0,1,2, or 3')

    opts, args = p.parse_args(sys.argv[1:])

    if len(args) < 2:
        print('Specify a hostname or IP for your casper platform. And either cx|real to
            indicate the type of spectrometer design.\n'
              'Run with the -h flag to see all options.')
        exit()
    else:
        hostname = args[0]
        mode_str = args[1]

        if mode_str=='cx':
            mode = 1
        elif mode_str=='real':
            mode = 0
        else:
            print('operation mode not recognized, must be "cx" or "real"')
            exit()

    if opts.fpgfile != '':
        bitstream = opts.fpgfile
    else:
        if mode == 1:
            fpg_prebuilt = '../prebuilt/rfsoc4x2/rfsoc4x2_tut_spec_cx.fpg'
        else:
            fpg_prebuilt = '../prebuilt/rfsoc4x2/rfsoc4x2_tut_spec.fpg'
        print('using prebuilt fpg file at %s' % fpg_prebuilt)
        bitstream = fpg_prebuilt

    if opts.adc_chan_sel < 0 or opts.adc_chan_sel > 3:
        print('adc select must be 0,1,2, or 3')
        exit()

    print('Connecting to %s...' % (hostname))
    fpga = casperfpga.CasperFpga(hostname)
    time.sleep(0.2)

    if not opts.skip:
        print('Programming FPGA with %s...' % bitstream)
        fpga.upload_to_ram_and_program(bitstream)
        print('done')
    else:
        fpga.get_system_information()
        print('skip programming fpga...')

    rfdc_fpga = fpga.adcs['rfdc']
    print('rfdc_fpga.init():', rfdc_fpga.init())
    print(rfdc_fpga.status())

    import second_part2
```

```
134     fpga.write('shared_bram', second_part2.buf)
135     print('done')
136 
137     fpga.write_int('wf_en', 1)
138 
139     time.sleep(0.1)
140 
141     print('Configuring accumulation period...')
142     fpga.write_int('acc_len', opts.acc_len)
143     print('done')
144 
145     print('setting capture on adc port %d' % opts.adc_chan_sel)
146     fpga.write_int('adc_chan_sel', opts.adc_chan_sel)
147     time.sleep(0.1)
148     print('done')
149 
150     print('Resetting counters...')
151     fpga.write_int('cnt_rst', 1)
152     fpga.write_int('cnt_rst', 0)
153     time.sleep(5)
154     print('done')
155 
156     try:
157         plot_spectrum(fpga, cx=mode)
158     except KeyboardInterrupt:
159         exit()
```

References

1. D. Price, "Spectrometers and Polyphase Filterbanks in Radio Astronomy," book chapter, 2016 (arXiv:1607.03579).
2. M. Bagchi, *Advances in Microwave and Sub-mm Astronomical Instrumentation and Analysis*, MSc Thesis, Queen's University, 2023 (Chapter 4: RFSoC-based backend for BVEX).
3. CASPER Collaboration, CASPER Toolflow Documentation, https://casper-toolflow.readthedocs.io/.